ACP turns repo automation into an operator-friendly system.

The package stages a shared runtime, installed profiles live outside the package, the heartbeat loop owns workflow control, and reconcile scripts own the final GitHub-facing outcome.

PACKAGE ROLE
Distribution + bootstrap

RUNTIME ROLE
Supervisor + scheduler + state

WORKER ROLE
Produce code + result artifacts

HOST WORKFLOW ROLE
Interpret, publish, recover

1. PACKAGE

CLI and staged shared-home

- **agent-control-plane.js** is the public entrypoint.
- **npx** runs against a staged copy, not a temp cache forever.
- **sync** publishes the durable runtime into **/agent-runtime**.

->

2. PROFILES

Installed profile registry

- Each repo gets one explicit profile identity.
- Config, prompts, and notes live outside the package.
- Profiles make ACP reusable across many repos.

->

3. RUNTIME

Supervisor and heartbeat

- **runtime start** spins up a long-lived supervisor.
- The supervisor re-invokes profile bootstrap safely.
- The shared heartbeat loop decides what to do next.

->

4. WORKERS

Issue and PR sessions

- ACP opens worktrees, renders prompts, and routes backends.
- Codex, Claude, and OpenClaw fit the same host contract.
- Workers emit artifacts instead of owning all policy.

->

5. OUTCOMES

Reconcile, GitHub, dashboard

- Reconcile scripts update labels, comments, cooldowns, and history.
- The dashboard reads state instead of guessing from terminals.
- Operators get visibility without becoming the

Key idea: ACP keeps intelligence in the workflow around the worker, not only inside the worker CLI.

After `runtime start`, ACP behaves like an operator loop.

The runtime path is designed so the shared scheduler owns concurrency, cooldowns, queue order, recurring lanes, and safe relaunch behavior.

1 Start a profile runtime

project-runtimectl.sh selects one installed profile and launches a dedicated supervisor for it.

2 Keep the runtime alive

project-runtime-supervisor.sh loops over profile bootstrap so the runtime can recover across passes instead of being a one-shot process.

3 Bootstrap safely

project-launchd-bootstrap.sh syncs runtime copies when needed, loads profile env, and invokes the published heartbeat script.

4 Run the shared heartbeat

heartbeat-safe-auto.sh handles locks, quota preflight, audits, and then forwards into the shared scheduler loop.

5 Select work by lane

agent-project-heartbeat-loop reconciles finished sessions, computes capacity, and chooses issue or PR flows for this pass.

Control boundary: worker quality matters, but the scheduler is where reliability is enforced.

Shared control

What the scheduler decides

- max worker concurrency
- heavy-worker limits
- provider cooldown waits
- blocked-recovery timing
- recurring and scheduled issue lanes

Workflow lanes

What can launch from one pass

- issue implementation
- issue scheduled checks
- issue blocked recovery
- PR review
- PR fix and merge repair

Why this matters

Workers can stay narrower.

ACP does not require every worker backend to understand

ACP treats worker execution and final outcome as separate phases.

That split is what lets the host workflow recover from weaker workers, stale state, or provider hiccups without hiding what happened.

ACP launch

1. Pick and prepare work

1A

Select issue or PR

The heartbeat chooses an eligible lane and launch target.

1B

Open or reuse worktree

ACP creates isolated workspaces and enforces guardrails around managed git state.

1C

Render prompt and context

Templates and issue/PR context become a stable input contract for the worker.

Backend session

2. Execute through an adapter

2A

Route backend

run-codex-task.sh forwards the same host contract into Codex, Claude, or OpenClaw wrappers.

2B

Produce artifacts

Workers leave behind **run.env**, **runner.env**, **result.env**, comments, and verification logs.

2C

Do not own all policy

The backend can focus on work, while host-side reconcile interprets what those artifacts mean.

Host reconcile

3. Interpret and publish

3A

Normalize final status

Reconcile scripts classify success, blocked, failure reason, cooldown, and resident metadata.

3B

Touch GitHub safely

Labels, comments, PR publication, and follow-up actions happen in one host-controlled place.

3C

Archive and observe

Runs move into history, state stays queryable, and the dashboard reflects the new reality.

Host contract: workers produce evidence; ACP decides the durable operational outcome.

The dashboard is read-only visibility over real scheduler state.

ACP keeps durable state in predictable places, then builds the dashboard from those sources instead of inventing a second control path.

Profile registry

Installed repo identities

Per-repo config and notes live outside the package, so upgrades do not overwrite local operator decisions.

```
~/.agent-runtime/control-plane/profiles/<id>/control-plane.yaml
```

Runtime home

Published shared engine

The packaged skill is copied into runtime-home so the machine has one durable source of truth for execution.

```
~/.agent-runtime/runtime-home/skills/openclaw/agent-control-plane
```

Dashboard pipeline

How the UI gets its truth

server.py serves the browser and exposes **/api/snapshot.json**. That snapshot is built from profile registry data, worker status, flow config, runs, and state directories.

Snapshot inputs

- profile registry
- render-flow-config.sh
- agent-project-worker-status
- runs, state, retries, controllers

->

Snapshot payload

- active and recent runs
- resident controllers
- provider cooldowns
- runtime process status

Runs and state

Per-profile operational memory

Live runs, resident workers, retries, cooldowns, and pending launches sit under each profile's runtime root.

```
.../projects/<profile>/runs and
.../projects/<profile>/state
```

History

Past sessions stay inspectable

Archived run directories preserve what happened without keeping stale live state around forever.

```
.../projects/<profile>/history
```

Trust model

Visibility without drift

Because the dashboard only reads ACP state, it can stay an honest monitoring surface instead of becoming a second scheduler that might disagree with the real runtime.

Operator payoff: you do not have to reconstruct runtime reality from tmux panes, branch names, and filesystem archaeology.